On y est presque, ce silence de mort du terminal est la preuve flagrante que le pod Spark cherche le fichier Python sur le réseau (COS) mais ne le trouve pas.

Spark tourne à l'intérieur du cluster Kubernetes, pas sur ta machine DevX. Il ne peut donc pas lire un fichier qui serait juste posé dans ton dossier local DevX. **Il faut obligatoirement que ton script Python soit déposé sur ton bucket COS (S3) avant de lancer la commande.**

Voici le code propre et la méthode pas-à-pas pour tout connecter.

1. Le code de streaming_app.py

Copie ce code et sauvegarde-le dans un fichier nommé streaming_app.py (sur ton DevX ou ton PC).

⚠️ *Attention : Remplace TON_BUCKET par le vrai nom de ton bucket (celui auquel tes clés secrètes te donnent accès, par exemple bu002i004226 ou bu002i007101 selon tes anciens PoCs).*

Python

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, concat_ws, lit

from pyspark.sql.types import StructType, StructField, StringType, IntegerType


# 1. Initialisation de Spark

spark = SparkSession.builder \

    .appName("PoC-Streaming-Synchrone") \

    .getOrCreate()


# 2. Définition du schéma de tes données d'entrée

schema = StructType([

    StructField("id_dossier", StringType(), True),

    StructField("montant", IntegerType(), True),

    StructField("statut", StringType(), True)

])
```

```
# 3. Définition des chemins sur le COS (Remplacer TON_BUCKET)
bucket_name = "TON_BUCKET"
input_path = f"s3a://{bucket_name}/poc_streaming/input/"
output_path = f"s3a://{bucket_name}/poc_streaming/output/"
checkpoint_path = f"s3a://{bucket_name}/poc_streaming/checkpoints/"

print(f"### Démarrage de l'écoute sur : {input_path}")

# 4. Lecture du flux en continu (Micro-batching)
df_stream = spark.readStream \
    .format("csv") \
    .schema(schema) \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .load(input_path)

# 5. Traitement (transformation de la donnée)
df_processed = df_stream \
    .withColumn("unique_key", concat_ws("-", col("id_dossier"), lit("processed"))) \
    .withColumn("montant_double", col("montant") * 2)

# 6. Écriture du résultat en continu
print(f"### Écriture des résultats vers : {output_path}")
query = df_processed.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", output_path) \
    .option("checkpointLocation", checkpoint_path) \
```

```
.trigger(processingTime="2 seconds") \
.start()

# 7. Empêcher l'application de s'arrêter
query.awaitTermination()
```

---

2. Où le mettre et comment lier tout ça ?

Étape A : Uploader le script sur le COS (Le point bloquant !)

C'est ça qui bloque ton job actuellement. Tu dois utiliser ton outil habituel (WinSCP, Cyberduck, ou l'interface web IBM) pour uploader ce fichier streaming_app.py **directement dans ton bucket COS**.

Par exemple, dépose-le dans un dossier /scripts/ à la racine de ton bucket.

Étape B : Lier le script dans ta commande spark-submit

Maintenant que le fichier est sur le COS, tu dois indiquer à Spark où aller le chercher.

Dans ton script bash de déploiement (celui que tu lances depuis DevX), la toute dernière ligne (qui pointe vers l'application) doit correspondre **exactement** à l'endroit où tu viens de le déposer.

Par exemple, la fin de ta commande spark-submit doit ressembler à ça :

Bash

```
 --conf spark.kubernetes.executor.secretKeyRef.AWS_ACCESS_KEY_ID=dh-cos-internal-bucket-keys-secret:AWS_ACCESS_KEY_ID \
 --conf spark.kubernetes.executor.secretKeyRef.AWS_SECRET_ACCESS_KEY=dh-cos-internal-bucket-keys-secret:AWS_SECRET_ACCESS_KEY \
 s3a://TON_BUCKET/scripts/streaming_app.py
```

3. Le test final

1. Assure-toi que l'ancien pod "zombie" est bien supprimé (kubectl delete pods --all -n spark-a102567-dev-ef25bd0b).
2. Relance ton script submit_streaming.sh depuis DevX.
3. Affiche les logs du nouveau pod Driver : kubectl logs -f pod/<NOM_DU_NOUVEAU_POD> -n spark-a102567-dev-ef25bd0b

Cette fois, les clés COS (dh-cos-internal-bucket-keys-secret) vont permettre au Driver de se connecter à ton bucket, de trouver le fichier streaming_app.py que tu viens d'y uploader, de le télécharger et de l'exécuter. Tu verras apparaître dans tes logs ### Démarrage de l'écoute sur... !